

Multiplayer Game

1. Alice is playing a game that involves pairing apples with oranges. The game produces apples and oranges in separate baskets at random instants. For Alice to get points, she must **pair exactly 1 apple with 2 oranges** by picking them from the respective baskets. Each basket is protected by a semaphore; A for the apple basket and O for the orange basket. To access a basket, Alice must first acquire the corresponding semaphore.

Complete the code in the below figure to help Alice to play this game. You should only use the operations Wait(A), Wait(O), Signal(A) and Signal(O) in the boxes provided. You may use any number of these operations (including zero) in each of the boxes. **Your solution must be deadlock-free even in the presence of other players who may be executing a different code.**

```
1
// Pick 1 apple
2
If (atleast 2 oranges in basket) {
** Pick 2 oranges and pair with apple. **
3
} else {
4
** Drop 1 apple back in basket **
5
}
```

Multiplayer Game

1: Wait(A)

// Pick 1 apple

2: Signal(A)
Wait(O)

If (atleast 2 oranges in basket) {
** Pick 2 oranges and pair with apple. **

3: Signal(O)

} else {

4: Signal(O)
Wait(A)

** Drop 1 apple back in basket **

5: Signal(A)

}

1. First acquire an apple and release semaphore A.
 2. If there are at least 2 oranges available, then acquire them and release semaphore O.
 3. Else release semaphore O, drop the apple back and release semaphore A.
- **There is no deadlock, because the two semaphores are never acquired together (no nesting).**

Semaphore System Calls

2. Consider the following implementations of the system calls Wait() and Signal().

```
Wait (Semaphore S)
{
    S.value--;
    If (S.value < -1) {
        block();
    }
}
```

```
Signal (Semaphore S)
{
    S.value++;
    If (S.value <= 0) {
        wakeup();
    }
}
```

Suppose these system calls are guaranteed to be atomic and a Semaphore S in this system is used to protect critical sections as shown below. Assume S is initialized to 0. Explain if this solution ensures mutual exclusion and progress with justifications?

Process P_i:

```
While(1){
    Wait(S);
    Critical-section for Pi
    Signal(S);
    Remainder-section for Pi
}
```

Semaphore System Calls

- **It satisfies mutual exclusion.** The first process to execute `Wait()` will get access to the critical section and the value of `S` becomes -1 (since the condition check in `Wait()` is < -1). Thereafter, no other process can enter its critical section until the first process executes `Signal()`. Also, since `Wait()` and `Signal()` are executed in pairs in that order and the Semaphore is initialized to 0, its value can never be greater than 0.
- **It satisfies progress as well.** When the value of `S` increments from -1 to 0, `Signal()` will execute an unnecessary `wakeup()` although no process is waiting for the Semaphore. If the value of `S` is less than -1 , then processes are blocked, and any execution of `Signal()` will `wakeup()` a blocked process. So, no process is blocked while the Semaphore is available, i.e., its value is 0.

Semaphore Implementation

3. Describe how *Wait(S)* and *Signal(S)* of a semaphore can be implemented using a *TestAndSet* instruction, given the below semaphore structure definition.

Hint: *The semaphore value and the process queue L are shared variables among different processes when those processes access the semaphore.*

```
typedef struct {  
    int value;  
    struct process *L;  
} semaphore
```

Where is the Critical Section?

//Current implementation of Wait(S)

S.value--;

Critical section

if (S.value < 0) {

Remainder section

 block();

Add current process to S.L;

}

Put current process to waiting state;

//Current implementation of Signal(S)

S.value++;

if (S.value <= 0) {

 wakeup(P);

Remove a process P from S.L;

}

Add process P to ready queue;

Not entire block() and wakeup() functions are in the critical section. 6

Semaphore Implementation

Shared variable: Boolean lock=false;

wait(S)

while TestAndSet(lock);

S.value = S.value - 1;

if (S.value < 0) {

add current process to S.L;

lock = false;

Put current process to waiting state;

}

else lock = false;

Semaphore Implementation (Cont')

Signal(S)

```
while Test-and-Set(lock);  
  S.value = S.value + 1;  
  if (S.value <= 0) {  
    remove a process P from S.L;  
    lock = false;  
    Add process P to ready queue;  
  }  
  else lock = false;
```